

FRD

Functional Requirements Document (FRD)

AWS Glue Data Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based data pipeline that ingests customer and order data from S3, performs data quality operations, implements Slowly Changing Dimension Type 2 (SCD2) logic using Apache Hudi, and produces analytical aggregations for business intelligence purposes.

Scope

This FRD covers:

- Ingestion of customer and order CSV files from S3
- Data cleaning and deduplication
- Registration of datasets in AWS Glue Data Catalog
- SCD Type 2 implementation for order summary tracking
- Incremental processing logic triggered by customer updates
- Aggregation of customer spending metrics
- Storage of curated and analytics datasets in S3

This FRD does NOT cover:

- Infrastructure provisioning or IAM policies
- Scheduling or orchestration mechanisms
- Data retention policies
- Disaster recovery procedures

Assumptions

1. CSV files in the source S3 paths are well-formed and use standard delimiters (comma-separated)
2. The term "Null" (string) and NULL (actual null value) both represent missing data
3. Duplicate detection is based on all columns in each dataset
4. SCD2 "change" detection compares all customer and order attributes
5. The Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` already exists
6. Hudi table format is used for SCD2 implementation with default Hudi configurations

7. "Incremental logic" means the pipeline processes only changed customer records, not full reprocessing
8. Date fields in Order data are in a parseable format (e.g., YYYY-MM-DD or MM/DD/YYYY)
9. TotalAmount in aggregation is calculated as $\text{PricePerUnit} \times \text{Qty}$ summed per customer
10. AWS Glue job has appropriate permissions to read from source buckets and write to target buckets
-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Files from S3
- Description:

The system shall read customer data in CSV format from the specified S3 location and load it into a structured dataset for downstream processing.

- Business Objective:

Enable the pipeline to access customer master data required for order enrichment and SCD2 tracking.

- Inputs:
 - Source: s3://adif-sdlc/sdlc_wizard/customerdata/
 - Format: CSV
 - Schema:
 - CustId: (datatype not specified in BRD)
 - Name: (datatype not specified in BRD)
 - EmailId: (datatype not specified in BRD)
 - Region: (datatype not specified in BRD)
- Processing Rules / Functional Steps:
 1. Connect to S3 path s3://adif-sdlc/sdlc_wizard/customerdata/
 2. Read all CSV files present in the path
 3. Parse CSV content into a structured dataset with columns: CustId, Name, EmailId, Region
 4. Preserve all records for subsequent cleaning operations
- Outputs:
 - In-memory dataset containing raw customer records
- Preconditions:
 - S3 path s3://adif-sdlc/sdlc_wizard/customerdata/ exists and contains at least one CSV file

- AWS Glue job has read permissions on the S3 bucket
- Postconditions:
- Customer dataset is available in memory for data cleaning
- Error / Validation Conditions:
- If S3 path does not exist, raise an error and halt processing
- If no CSV files are found, raise an error and halt processing
- If CSV structure does not match expected schema (missing columns), raise an error
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Files from S3
- Description:

The system shall read order transaction data in CSV format from the specified S3 location and load it into a structured dataset for downstream processing.

- Business Objective:

Enable the pipeline to access order transaction data required for customer order summary generation and aggregation.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Schema:
- OrderId: (datatype not specified in BRD)
- ItemName: (datatype not specified in BRD)
- PricePerUnit: (datatype not specified in BRD)
- Qty: (datatype not specified in BRD)
- Date: (datatype not specified in BRD)
- CustId: (datatype not specified in BRD)
- Processing Rules / Functional Steps:
- 1. Connect to S3 path s3://adif-sdlc/sdlc_wizard/orderdata/
- 2. Read all CSV files present in the path
- 3. Parse CSV content into a structured dataset with columns: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- 4. Preserve all records for subsequent cleaning operations

- Outputs:
- In-memory dataset containing raw order records
- Preconditions:
- S3 path s3://adif-sdlc/sdlc_wizard/orderdata/ exists and contains at least one CSV file
- AWS Glue job has read permissions on the S3 bucket
- Postconditions:
- Order dataset is available in memory for data cleaning
- Error / Validation Conditions:
- If S3 path does not exist, raise an error and halt processing
- If no CSV files are found, raise an error and halt processing
- If CSV structure does not match expected schema (missing columns), raise an error
-

FR-CLEAN-001: Remove Null Values and Duplicates from Customer Data

- Title: Clean Customer Dataset

- Description:

The system shall remove records containing null values (both string "Null" and actual NULL) and eliminate duplicate records from the customer dataset.

- Business Objective:

Ensure data quality by removing incomplete and redundant customer records before catalog registration and downstream processing.

- Inputs:
- Raw customer dataset from FR-INGEST-001
- Processing Rules / Functional Steps:
 1. Identify and remove all records where any column contains the string "Null" (case-insensitive) or actual NULL value
 2. Identify duplicate records by comparing all columns (CustId, Name, EmailId, Region)
 3. Retain only one instance of each unique record
 4. Preserve the cleaned dataset for catalog registration
- Outputs:
- Cleaned customer dataset with no null values and no duplicates
- Preconditions:

- Raw customer dataset has been successfully loaded (FR-INGEST-001 completed)
- Postconditions:
- Customer dataset contains only complete, unique records
- Error / Validation Conditions:
- If all records are removed due to null values, log a warning but continue processing with an empty dataset
-

FR-CLEAN-002: Remove Null Values and Duplicates from Order Data

- Title: Clean Order Dataset

- Description:

The system shall remove records containing null values (both string "Null" and actual NULL) and eliminate duplicate records from the order dataset.

- Business Objective:

Ensure data quality by removing incomplete and redundant order records before catalog registration and downstream processing.

- Inputs:

- Raw order dataset from FR-INGEST-002

- Processing Rules / Functional Steps:

1. Identify and remove all records where any column contains the string "Null" (case-insensitive) or actual NULL value
2. Identify duplicate records by comparing all columns (OrderId, ItemName, PricePerUnit, Qty, Date, CustId)
3. Retain only one instance of each unique record
4. Preserve the cleaned dataset for catalog registration

- Outputs:

- Cleaned order dataset with no null values and no duplicates

- Preconditions:

- Raw order dataset has been successfully loaded (FR-INGEST-002 completed)

- Postconditions:

- Order dataset contains only complete, unique records

- Error / Validation Conditions:

- If all records are removed due to null values, log a warning but continue processing with an empty dataset

-

FR-CATALOG-001: Register Customer Data in Glue Data Catalog

- Title: Create Glue Catalog Table for Customer Data

- Description:

The system shall register the cleaned customer dataset as a table in the AWS Glue Data Catalog under the specified database and table name.

- Business Objective:

Make customer data discoverable and queryable through AWS services such as Athena and enable metadata-driven processing.

- Inputs:

- Cleaned customer dataset from FR-CLEAN-001
- Target database: gen_ai_poc_databrickscoe
- Target table: sdlc_wizard_customer

- Processing Rules / Functional Steps:

1. Write cleaned customer dataset to a persistent storage location (format and location not specified in BRD)
2. Register the dataset as table `sdlc\_wizard\_customer` in database `gen\_ai\_poc\_databrickscoe`
3. Ensure table schema matches the customer schema: CustId, Name, EmailId, Region
4. Update catalog metadata to reflect current data location and schema

- Outputs:

- Glue Data Catalog table: gen_ai_poc_databrickscoe.sdlc_wizard_customer

- Preconditions:

- Database `gen\_ai\_poc\_databrickscoe` exists in Glue Data Catalog
- Customer data has been cleaned (FR-CLEAN-001 completed)

- Postconditions:

- Table `sdlc\_wizard\_customer` is queryable via Athena and accessible by other AWS Glue jobs

- Error / Validation Conditions:

- If database does not exist, raise an error and halt processing
- If table registration fails, raise an error and halt processing

-

FR-CATALOG-002: Register Order Data in Glue Data Catalog

- Title: Create Glue Catalog Table for Order Data
- Description:

The system shall register the cleaned order dataset as a table in the AWS Glue Data Catalog under the specified database and table name.

- Business Objective:

Make order data discoverable and queryable through AWS services such as Athena and enable metadata-driven processing.

- Inputs:

- Cleaned order dataset from FR-CLEAN-002
- Target database: gen_ai_poc_databrickscoe
- Target table: sdlc_wizard_order

- Processing Rules / Functional Steps:

1. Write cleaned order dataset to a persistent storage location (format and location not specified in BRD)
2. Register the dataset as table `sdlc\_wizard\_order` in database `gen\_ai\_poc\_databrickscoe`
3. Ensure table schema matches the order schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
4. Update catalog metadata to reflect current data location and schema

- Outputs:

- Glue Data Catalog table: gen_ai_poc_databrickscoe.sdlc_wizard_order

- Preconditions:

- Database `gen\_ai\_poc\_databrickscoe` exists in Glue Data Catalog
- Order data has been cleaned (FR-CLEAN-002 completed)

- Postconditions:

- Table `sdlc\_wizard\_order` is queryable via Athena and accessible by other AWS Glue jobs

- Error / Validation Conditions:

- If database does not exist, raise an error and halt processing
- If table registration fails, raise an error and halt processing

-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Maintain Order Summary with SCD Type 2 Logic Using Hudi

- Description:

The system shall join customer and order data, apply Slowly Changing Dimension Type 2 logic to track historical changes, and store the result in a Hudi-formatted table on S3. When customer or order data changes, the system shall insert new active records and close previous records by setting IsActive to false and updating EndDate.

- Business Objective:

Maintain a complete historical record of customer order summaries, enabling time-based analysis and audit trails.

- Inputs:

- Cleaned customer dataset from FR-CLEAN-001
- Cleaned order dataset from FR-CLEAN-002
- Existing ordersummary Hudi table (if present) from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

- Processing Rules / Functional Steps:

1. Perform an inner join between customer and order datasets on CustId
2. For each joined record, compare against existing active records in the ordersummary table
3. If a matching record exists (based on key columns) and any attribute has changed:
 - Set IsActive = false for the existing record
 - Set EndDate = current processing timestamp for the existing record
- Insert a new record with IsActive = true, StartDate = current processing timestamp, EndDate = null
4. If no matching record exists, insert a new record with IsActive = true, StartDate = current processing timestamp, EndDate = null
5. Set OpTs (operation timestamp) to the current processing timestamp for all new/updated records
6. Write the updated dataset to s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ in Hudi format
7. Register or update the table `ordersummary` in database `gen\_ai\_poc\_databrickscoe`

- Outputs:

- Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Glue Data Catalog table: gen_ai_poc_databrickscoe.ordersummary
- Table includes columns: (customer columns + order columns + IsActive, StartDate, EndDate, OpTs)

- Preconditions:

- Customer and order data have been cleaned and are available
- S3 path s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ is accessible
- Database `gen\_ai\_poc\_databrickscoe` exists

- Postconditions:
- All current order summaries are marked as active (IsActive = true)
- All superseded order summaries are marked as inactive (IsActive = false) with appropriate EndDate
- Historical changes are preserved
- Error / Validation Conditions:
- If join produces no records, log a warning but complete the process
- If Hudi write fails, raise an error and halt processing
- If catalog update fails, raise an error
-

FR-INCR-001: Process Incremental Customer Updates

- Title: Trigger SCD2 Updates Based on Customer Changes
- Description:

The system shall identify customer records that have changed since the last processing run and trigger SCD2 updates in the ordersummary table only for affected customers.

- Business Objective:

Optimize processing performance by updating only the order summaries affected by customer data changes, rather than reprocessing the entire dataset.

- Inputs:
- Current customer dataset from FR-CLEAN-001
- Previous customer dataset (from prior run or baseline)
- Existing ordersummary Hudi table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Processing Rules / Functional Steps:
 1. Compare current customer dataset against previous customer dataset to identify changed records (based on CustId as key)
 2. Identify all CustId values that have changed
 3. Filter order data to include only orders associated with changed CustId values
 4. Execute SCD2 logic (FR-SCD2-001) only for the filtered customer-order subset
 5. Merge the updated records into the existing ordersummary Hudi table
- Outputs:
- Updated ordersummary Hudi table with incremental changes applied
- Preconditions:

- Previous customer dataset baseline exists
- FR-SCD2-001 logic is implemented and functional
- Postconditions:
 - Only order summaries for changed customers are updated
 - Unchanged order summaries remain intact
- Error / Validation Conditions:
 - If no customer changes are detected, skip SCD2 processing and log informational message
 - If previous baseline does not exist, treat all records as new and process accordingly
-

FR-AGG-001: Generate Customer Aggregate Spend

- Title: Create Customer Spending Aggregation Table
- Description:

The system shall calculate total spending per customer by aggregating order amounts and store the result in a dedicated analytics table on S3.

- Business Objective:

Provide a pre-aggregated view of customer spending to support business intelligence and reporting use cases.

- Inputs:
 - Cleaned customer dataset from FR-CLEAN-001
 - Cleaned order dataset from FR-CLEAN-002
- Processing Rules / Functional Steps:
 1. Join customer and order datasets on CustId
 2. Calculate TotalAmount for each order as: $\text{PricePerUnit} \times \text{Qty}$
 3. Group by customer Name and Date
 4. Sum TotalAmount for each group
 5. Select columns: Name, TotalAmount, Date
 6. Write the aggregated dataset to s3://adif-sdlc/analytics/customeragggregatespend/
 7. Register or update the table `customeragggregatespend` in database `gen\_ai\_poc\_databrickscoe`
- Outputs:
 - Analytics table at s3://adif-sdlc/analytics/customeragggregatespend/
 - Glue Data Catalog table: gen_ai_poc_databrickscoe.customeragggregatespend
 - Schema: Name, TotalAmount, Date

- Preconditions:
- Customer and order data have been cleaned and are available
- S3 path s3://adif-sdlc/analytics/customeraggregatespend/ is accessible
- Database `gen\_ai\_poc\_databrickscoe` exists
- Postconditions:
- Aggregated spending data is available for querying via Athena
- Data reflects the most recent customer and order datasets
- Error / Validation Conditions:
- If join produces no records, create an empty table and log a warning
- If aggregation calculation fails (e.g., non-numeric PricePerUnit or Qty), raise an error
- If write to S3 fails, raise an error and halt processing
-

3. Non-Functional Considerations

Performance

- The pipeline should leverage AWS Glue's distributed Spark engine to handle large datasets efficiently
- Incremental processing (FR-INCR-001) should reduce processing time compared to full reprocessing

Scalability

- The solution should scale horizontally as data volume grows by adjusting Glue DPU allocation

Monitoring and Observability

- AWS CloudWatch should be used to monitor Glue job execution, track failures, and log processing metrics

Data Governance

- AWS Lake Formation may optionally be used to enforce fine-grained access control and data governance policies

Query Performance

- AWS Athena should be used to query curated and analytics tables
- Hudi table format enables efficient upserts and time-travel queries
-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Description	
-----	-----	-----	
FR-INGEST-001	Source Input (AWS S3) - Customer CSV	Ingestion of customer data from s3://adif-sdlc/sdlc_wizard/customerdata/	
FR-INGEST-002	Source Input (AWS S3) - Order CSV	Ingestion of order data from s3://adif-sdlc/sdlc_wizard/orderdata/	
FR-CLEAN-001	Data Cleaning	Removal of null values and duplicates from customer dataset	
FR-CLEAN-002	Data Cleaning	Removal of null values and duplicates from order dataset	
FR-CATALOG-001	Glue Catalog Tables	Registration of customer data as sdlc_wizard_customer	
FR-CATALOG-002	Glue Catalog Tables	Registration of order data as sdlc_wizard_order	
FR-SCD2-001	SCD Type 2 (Hudi on S3)	Implementation of SCD2 logic for ordersummary table	
FR-INCR-001	Incremental Logic	Processing only changed customer records to update ordersummary	
FR-AGG-001	Aggregation Table	Generation of customeraggregatespend analytics table	

- Cross-Requirement Dependencies:
- FR-CLEAN-001 depends on FR-INGEST-001
- FR-CLEAN-002 depends on FR-INGEST-002
- FR-CATALOG-001 depends on FR-CLEAN-001
- FR-CATALOG-002 depends on FR-CLEAN-002
- FR-SCD2-001 depends on FR-CLEAN-001 and FR-CLEAN-002
- FR-INCR-001 depends on FR-SCD2-001
- FR-AGG-001 depends on FR-CLEAN-001 and FR-CLEAN-002
-
- End of Functional Requirements Document